

Algorithm Design Paradigms

Handout on Geometric DnC

05 February 2026

As a teacher of economics, I have always believed that if you are not able to explain the economy in a language young people can understand, then, quite simply, you are clueless yourself.¹

— Yanis Varoufakis (*Talking to My Daughter About the Economy*)

§1 Recap

You might have already seen **Merge Sort** and **Quick Sort** in the class. You probably also saw **Quick Select** (or median finding). Let's look at the psudo-code for these just to come to the same page.

MERGE SORT

```
1 def MSORT(list):
2     |   L = left half of list
3     |   R = rest of the list
4     |   return MERGE(MSORT(L), MSORT(R))
5 def MERGE(as, bs):
6     |   if (empty as):
7     |       |   return bs
8     |   elif (empty bs):
9     |       |   return as
10    |   else:
11    |       |   if as[0] <= bs[0]:
12    |       |       |   a[0] : MERGE(tail as, bs)
13    |       |       |   else:
14    |       |       |   b[0] : MERGE(as, tail bs)
```

Note, as primarily a Haskell person, my psudo code borrows some stuff from Haskell. In this case, `x:xs` is a stand in for `append(x, xs)` and `tail xs` refers to `a[1..]`.

The time complexity comes from the recursion $T(n) = 2T(\frac{n}{2}) + O(n) \Rightarrow T(n) = O(n \log n)$.

Correctness is proven in 2 steps:

1. Prove that given 2 sorted lists `as`, `bs`, `MERGE as bs` is sorted by induction
2. Prove the correctness of Merge Sort by induction.

¹This is a lovely book which everyone should read.

QUICK SORT

```

1  def QSORT(list):
2      pivot = random element from list
3      L = items less or equal to pivot in the (list - pivot)
4      R = items greater than the pivot in (list - pivot)
5      return QSORT(L) ++ [pivot] ++ QSORT(R)

```

Here the correctness follows swiftly via induction. However, the time complexity is not that clear.

In the worst case scenario, we could keep splitting the list into a length 0 and a length $n - 1$ list.
 $T(n) = T(0) + T(n - 1) + O(n) \Rightarrow T(n) = O(n^2)$.

Then why are we interested in Quick Sort? The reason is, that it has expected time complexity of $O(n \log n)$.

Wrong Proof. Notice,

$$\begin{aligned}
 T(n) &= T(L) + T(R) + O(n) \text{ where } L, R \text{ are lengths of left and right lists} \\
 \Rightarrow \mathbb{E}(T(N)) &= \mathbb{E}(T(L) + T(R) + O(n)) = \mathbb{E}(T(L)) + \mathbb{E}(T(R)) + \mathbb{E}(O(n)) \text{ by linearity of expectation} \\
 &= \mathbb{E}(T(L)) + \mathbb{E}(T(R)) + O(n) \\
 &= T(\mathbb{E}(L)) + T(\mathbb{E}(R)) + O(n) \\
 &= T\left(\frac{n}{2}\right) + T\left(\frac{n}{2}\right) + O(n) \\
 &\Rightarrow \mathbb{E}(T(L)) = O(n \log n)
 \end{aligned}$$

□

What is the error here?²

The actual way we prove this is as follows:

Proof. Notice, our time complexity here is just the number of times we compare elements.

Consider the list we are sorting a permutation of $[x_1, x_2, \dots, x_n]$. Let $X_{i,j}$ be a random variable which is 1 if the x_i and x_j are compared and 0 otherwise. Let $p_{i,j}$ be the probability that x_i and x_j are compared. Then, $\mathbb{E}(X_{i,j}) = 1 * p + 0 * (1 - p) = p$.

Using the linearity of expectation, we can say $\mathbb{E}(T(n)) = \sum_{i,j} \mathbb{E}(X_{i,j}) = \sum_{i,j} p_{i,j}$.

We rewrite this summation as

$$\begin{aligned}
 \mathbb{E}(T(n)) &= \sum_{i,j} p_{i,j} \\
 &= \sum_{i=1}^n \sum_{j=i+1}^n p_{i,j}
 \end{aligned}$$

Despite a toothy appearance, this is rather easy and elegant way to actually compute $p_{i,j}$.

Notice that each element in the array (except the pivot) is compared only to the pivot at each level of the recurrence. To compute $p_{i,j}$, we shift our focus to the elements $[x_i, x_{i+1}, \dots, x_j]$. If this is split into

²This is a very, very common erroneous proof. It won't be hard to find DSA books with this kind of an error

two parts, x_i and x_j can no longer be compared. Hence, x_i and x_j are compared only when from the first pivot from the range $[x_i, x_{i+1}, \dots, x_j]$ is either x_i or x_j .

This clearly has probability $p_{i,j} = \frac{1}{j-i+1} + \frac{1}{j-i+1} = \frac{2}{j-i+1}$. Thus,

$$\begin{aligned}\mathbb{E}(T(n)) &= \sum_{i=1}^n \sum_{j=i+1}^n \frac{2}{j-i+1} \\ &= \sum_{i=1}^n 2 \left(\frac{1}{2} + \dots + \frac{1}{n-i+1} \right) \\ &= 2 \sum_{i=1}^n \left(1 + \frac{1}{2} + \dots + \frac{1}{n-i+1} - 1 \right) \\ &\leq \sum_{i=1}^n O(\log(n)) \\ &\leq O(n \log(n))\end{aligned}$$

And we are done. □

Remember, $\mathbb{E}(T(n)) \neq T(\mathbb{E}(n))$ in general. We can't willy nilly switch symbols around and hope it works out.

Finally,

QUICK SELECT

```

1  def QSELECT(arr, k):
2      n = LENGTH(arr)
3      pivot = choose random element of arr
4      L = items less or equal to pivot in the (list - pivot)
5      R = items greater than the pivot in (list - pivot)
6      l = LENGTH(L)
7      if l == k-1:
8          return pivot
9      elif l < k-1:
10         return QSELECT(R, k - l - 1)
11     else:
12         return QSELECT(L, k)
```

Again, the worst case time complexity is quite easy to judge:

$$\begin{aligned}T(n) &= T(n-1) + T(0) + O(n) \\ &\Rightarrow T(n) = O(n^2)\end{aligned}$$

But if we redo the above analysis for expected time complexity we would get (other than lots of pain) is that the expected time complexity is $O(n)$.

However, given that there is a chance of our $O(n)$ algorithm running in $O(n^2)$, we can try to choose a better pivot.

BETTER PIVOT

```

1 def PIVOT(arr):
2     a1, a2, a3, a4, a5 = array's of size  $\frac{|arr|}{5}$ 
3     m1, m2, m3, m4, m5 = median of a1, a2, a3, a4, a5
4     // This is a comment!
5     // We find median via MSELECT which we will define below
6     return (median [m_1, m_2, m_3, m_4, m_5])
7 def MSELECT(arr, k):
8     n = LENGTH(arr)
9     pivot = PIVOT(arr)
10    L = items less or equal to pivot in the (list - pivot)
11    R = items greater than the pivot in (list - pivot)
12    l = LENGTH(L)
13    if l == k-1:
14        return pivot
15    elif l < k-1:
16        return QSELECT(R, k - l - 1)
17    else:
18        return QSELECT(L, k)

```

We can show that our scheme chooses a pivot such that $\frac{3n}{10} < L, R < \frac{7n}{10}$.

The time complexity hence is

$$T(n) \leq T\left(\frac{7n}{10}\right) + 5T\left(\frac{n}{5}\right) + O(n)$$

And that is literally the recurrence I started the class with.

The goal of this rather lengthy recap was to convince you **Quick** implies easy to code and prove the correctness of but the time complexity is a nightmare; while in case of **Merge**, while we have to work to code and show correctness, the time complexity is normally a cakewalk.

§2 Closest Point

? Problem

Given n points on the plane. Each point p_i is defined by its coordinates (x_i, y_i) . We want to find among them two such points, such that the (Euclidean) distance between them is minimal.

There is an obvious $O(n^2)$ solution. We will use divide and contour to find a $O(n \log n)$ solution.

💡 Idea

Given our goal is $O(n \log(n))$, we can sort our points (to begin with) without any extra cost

Let's sort our points by lexicographically.

Once we sort our points, we can draw a vertical line l at some x such that the points are split in left and right halves, L and R .

Let's say we find the minimum distance d_L and d_R in the halves. We now want a way to combine. Notice, the obvious idea doesn't work as we $T(n) = 2T(\frac{n}{2}) + O(\frac{n}{2} * \frac{n}{2})$ is $O(n^2)$.

We make the following observation, if $d = \min(d_L, d_R)$ then if there are points across the partition that are closer than d , they must be within d of the partition.

This restricts our search to a strip of width $2d$ centered at l . But unfortunately, this is not enough as a lot of points could be here.

We make a similar observation about the y -axis now. For a point in this strip, if it's distance is less than d , then the difference in y coordinates can't be more than d .

This implies that for each point in the strip, we only check in a $2d \times d$ rectangle around it. While it may not seem like it, we are done. Why?

By the fact that no two points in L are closer than d and similarly for R , we can have at most 8 points in such a rectangle.

This implies, $T(n) < 2T(\frac{n}{2}) + 8n + O(n) = 2T(\frac{n}{2}) + O(n) \Rightarrow T(n) = O(n \log n)$.

And we are done!

NEAREST POINTS

```

1  def CLOSEST(points):
2      n = LENGTH(points)
3      if n <= 3:
4          return BRUTE_FORCE(points)
5      mid = n / 2
6      L = left half of points
7      R = right half of points
8      d_L = CLOSEST(L)
9      d_R = CLOSEST(R)
10     d = min(d_L, d_R)
11     strip = points whose x-distance from mid-line <= d
12     strip = strip sorted by y-coordinate
13     return min(d, STRIP_CLOSEST(strip, d))
14 def STRIP_CLOSEST(strip, d):
15     m = LENGTH(strip)
16     min_d = d
17     for i = 0 to m-1:
18         for j = i+1 to m-1:
19             if strip[j].y - strip[i].y >= min_d:
20                 break
21             min_d = min(min_d, DIST(strip[i], strip[j]))
22     return min_d

```

NEAREST POINTS

```

23 def BRUTE_FORCE(points):
24     min_d = infinity
25     for i = 0 to LENGTH(points)-1:
26         for j = i+1 to LENGTH(points)-1:
27             min_d = min(min_d, DIST(points[i], points[j]))
28     return min_d

```

Notice that while the design and implementation of the algorithm took some work, the analysis was a freebie.

What is the “Quick” version of this? The idea is to divide the space of points into squares of some side d and brute force the comparison of points in a square and points in neighboring squares.

If we had k squares and each square had some n_i points such that $\sum_{i=1}^k n_i = n$; then the number of comparisons is:

$$\begin{aligned}
 & \sum_{i=1}^k \left(n_i^2 + \sum_{j \in N(i)} n_i n_j \right) \\
 & < \sum_{i=1}^k \left(n_i^2 + \sum_{j \in N(i)} \max(n_i, n_j)^2 \right) \\
 & < \sum_{i=1}^k 9n_i^2 \quad \text{as every cell can have atmost 8 neighbors} \\
 & = 9 \sum_{i=1}^k n_i^2 \\
 & = \Theta \left(\sum_{i=1}^k n_i^2 \right)
 \end{aligned}$$

Notice, if our d is close to the actual smallest distance, then the algorithm is linear time. Although, how do we guess this?

Similar to Quick Select, we make an approximate guess. Here, one way of doing so is by choosing some random $\sqrt{n}$ points and take their shortest distance (by brute force).

QUICK NEAREST POINTS

```

1 def QUICK_DIST(points):
2     n = LENGTH(points)
3     if n <= 3:
4         return BRUTE_FORCE(points)
5     sample_size = CEIL(sqrt(n))
6     S = RANDOM_SAMPLE(points, sample_size)
7     d = BRUTE_FORCE(S)
8     grid = EMPTY_HASH_MAP
9     cell_size = d

```

QUICK NEAREST POINTS

```

10   for each p in points:
11       cx = FLOOR(p.x / cell_size)
12       cy = FLOOR(p.y / cell_size)
13       INSERT p INTO grid[(cx, cy)]
14       min_d = d
15       for each cell (cx, cy) in grid:
16           C = grid[(cx, cy)]
17           for each point p in C:
18               for dx = -1 to 1:
19                   for dy = -1 to 1:
20                       if (cx + dx, cy + dy) EXISTS in grid:
21                           for each point q in grid[(cx + dx, cy + dy)]:
22                               if p != q:
23                                   min_d = min(min_d, DIST(p, q))
24   return min_d
25 def BRUTE_FORCE(points):
26     min_d = infinity
27     for i = 0 to LENGTH(points)-1:
28         for j = i+1 to LENGTH(points)-1:
29             min_d = min(min_d, DIST(points[i], points[j]))
30   return min_d

```

Proof. Notice, we atmost consider the points 2 diagonals away from points. Consider $0 < x, y < s$ (we will later take the limit of $s \rightarrow \infty$).

This tells that, given some d , we make atmost:

$$\begin{aligned}
 & \left(\frac{s}{d}\right)^2 \left(\frac{8\pi d^2}{s^2} n\right)^2 \\
 &= \frac{s^2}{d^2} \frac{64\pi^2 d^4}{s^4} n^2 \\
 &= \frac{64\pi^2 d^2}{s^2} n^2
 \end{aligned}$$

Now notice, $d = \mathbb{E}(\text{minimum distance of } \sqrt{n} \text{ random points in a square of side } s)$.

We will use $\mathbb{E}(X) = \int_0^\infty \mathbb{P}(X > x) dx$ for a positive support varable X to get a crude upper bound on d .

Notice, if we already have put some k points on the disk, the odds of the next point being atleast x away is atmost

$$\begin{aligned}
& 1 - k \frac{\pi \left(\frac{x}{2}\right)^2}{s^2} \\
& = 1 - \frac{k\pi x^2}{16s^2}
\end{aligned}$$

This comes from putting a disk of radius $\frac{x}{2}$ at each of the previous points. These disks can't overlap, and at least a quarter of each disk is inside the square.

We can now integrate

$$\begin{aligned}
\mathbb{E}(X) & < \int_0^{\sqrt{2}} \prod_{i=0}^{\sqrt{n}-1} \left(1 - \frac{k\pi x^2}{16s^2}\right) dx \\
& < \int_0^{\sqrt{2}} \prod_{i=0}^{\sqrt{n}-1} \left(1 - \frac{\pi x^2}{16s^2}\right)^k dx \\
& < \int_0^{\sqrt{2}} \left(1 - \frac{\pi x^2}{16s^2}\right)^{\frac{\sqrt{n}(\sqrt{n}-1)}{2}} dx
\end{aligned}$$

Now Mathematica evaluates this integral as $\frac{2\sqrt{2}s}{\sqrt{n}}$.

We can plug this in to get the expected time complexity as $512\pi^2 n = \Theta(n)$. □

i Remark

The bounds used are very, very blunt. In practice, the constant in my testing were close to 10.

The choice to use these bounds was made as these are the 'easiest'. Bounding it better would require more probability (trick|theo)ry than I am comfortable introducing.

§3 Convex Hull

Another example of a famous divide and conquer algorithm is convex hull.

? Problem

Finding the smallest convex polygon that encloses a given set of 2D points

A simple approach probably would be thinking of this as wrapping a gift. Start at the right most, bottom point p_1 (which will definitely be on the hull) and then check other points to see if a p_i is such that all points will be to one side of the line, if yes; then add p_i to the hull and continue onwards.

This is perhaps the first example of output dependent algorithm you'll see. We make $O(n)$ comparisons every iteration and we have h iterations where h is the number of points on the hull. This makes the complexity $O(nh)$.

We can prove (with lots of difficulty, see On the Expected Complexity of Random Convex Hulls) that $\mathbb{E}(h) \sim O\left(n^{\frac{1}{3}}\right)$ and hence, we have the expected complexity of Gift Wrapping as $O\left(n^{\frac{4}{3}}\right)$.

Note: The worst case is $O(n^2)$ as we could have all the n vertices on the same polygon.

What about a MergeHull algorithm? We can divide the points arbitrarily and make two hulls. Now how do we merge?

Looking at some of our examples, it is clear we want to find the upper tangent and lower tangent. While we could brute our way through it, and have an $O(n^2)$ merge step, that would make it no better (actually worse) as $T(n) = 2T(\frac{n}{2}) + O(n^2) \Rightarrow T(n) = O(n^2 \log n)$.

So what do we do? Not checking a number of useless combinations might seem like a good idea. Second is the following subroutine:

We start with the two points closest to our line of division on both the sides. We can now try to get better by simply seeing the direction the angle goes in if we move clockwise or anticlockwise in the second hull. Convince yourself that we make atmost $2n$ updates. Similarly, we do the same to get the down tangent.

And with that, we get $T(n) = 2T(\frac{n}{2}) + O(n) \Rightarrow T(n) = O(n \log n)$

Notice, I am omitting the proof here as it is painful, it is the usual induction but painful. The psudo-code has been skipped for the sake of brevity.

What about QuickHull? Similer to how we found the down-left point (which has to be on the hull) we also identify the up-right point (which will also be on the hull). Draw a line between them.

Now find the points on either sides that has the maximum perpendicular distance from this line and make the triangle. Get rid of points on the inside of the triangles and repeat this triangulation. We are done!

The proof of correctness is obvious. The implementation seems obvious. The time complexity?

Conjecture 3.1. *The time complexity of QuickHull is $O(n \log n)$*

Very recently (in 2024) a version of QuickHull (called Ray Shooting QuickHull) was proven to be in $O(n \log n)$ strengthening the conjecture, but the proof remains elusive.

And that's all for this tutorial. These notes have exactly 2026 words! Yay